Product Design Document

ZimDash

Harare's food, delivered — a multi-vendor delivery marketplace bringing restaurants and supermarkets into one ordering experience, built for the way Harare actually pays and gets around.

Status Prototype · Pre-launchPlatform iOS · Android · Web (single Flutter codebase)**Version** 1.0.0, build 1**Date** 26 August 2026

Contents

01

Overview

ZimDash is a food-and-grocery delivery app for Harare, Zimbabwe. Customers browse restaurants and supermarkets, order from a shared cart, and pay by EcoCash, card, or cash on delivery — the same on-demand pattern proven by platforms like DoorDash, adapted for a market those platforms don't yet serve.

Two things make it a Harare product rather than a reskinned import: the payment methods people actually use (EcoCash first, not an afterthought), and a visual identity drawn from the Zimbabwean flag and the flame lily rather than a generic delivery-app template. The current build is a fully designed, working prototype — every screen in the customer journey is implemented against a real Postgres backend. What's missing is the operational machinery — real payments, real dispatch, vendor and rider tooling — covered in Section 6.

02

Problem & opportunity

Most Harare restaurants that deliver today do it over WhatsApp or phone calls, with a rider dispatched ad hoc. That works, but it caps how many orders a kitchen can handle at once, gives customers no visibility into where their order is, and gives no restaurant a way to be discovered beyond word of mouth. Global delivery platforms haven't localized for Zimbabwe: they assume card-first payment, not EcoCash, and they don't carry the informal and traditional-cuisine vendors that make up a large share of how Harare actually eats.

Combining restaurants and supermarkets in one app is deliberate — it gives customers a reason to open ZimDash more than once a week, and gives the business two demand pools instead of one while the marketplace is still building density.

03

Target users

customers

Urban Harare residents ordering to home or work — Avondale, Borrowdale, Mount Pleasant, Belvedere, Greendale, and the CBD are the initial suburb set. Pay by EcoCash, card, or cash; want to see a fee and ETA before committing, and want to track an order once it's placed.

restaurants & supermarkets

Vendors who want an online ordering and delivery channel without building their own app or dispatch system. Range from established traditional-Zimbabwean kitchens to grocery outlets.

riders (not yet built)

Motorbike and bicycle couriers who accept jobs, navigate to pickup and drop-off, and get paid per delivery. No rider-facing surface exists yet — see Section 6.

ops / admin (not yet built)

The internal team approving vendors, managing promo codes, and handling disputes. No admin surface exists yet — currently all reference data is edited by hand in Supabase's SQL editor.

04

Value proposition

• **For customers:** one app for restaurants and groceries, delivery fee and time shown up front, checkout that takes EcoCash as naturally as cash, promo codes and tipping, and order tracking rather than a black box.

• **For vendors:** a new discovery and ordering channel with no upfront technology cost, once vendor tooling exists (Section 6).

• **For riders:** flexible, per-delivery earning, once the rider app exists (Section 6).

05

What's built today

The prototype implements the full customer journey end to end, backed by a real Supabase Postgres database with row-level security — not just static mock screens.

AreaWhat it does	
Browse & discover	Restaurant and supermarket list with search (name, cuisine, tag, menu item), category filter chips, favourites-only filter, and sort by relevance, rating, delivery fee, or speed.
Restaurant detail	Menu grouped by category, item detail sheet with quantity and notes, add-to-cart.
Cart & checkout	Multi-restaurant cart; delivery details form (name, phone, address, Harare suburb picker); payment method choice of EcoCash, card, or cash on delivery; promo code redemption; tip selection (15% default, quick-select percentages, or custom amount); full order summary (subtotal, delivery fee, service fee, tip, discount, total).
Order tracking	Four-stage progress view (Confirmed → Cooking → Rider on the way → Delivered) with an ETA and toast notification on delivery.
Order history	Past orders persisted per user and viewable in an Orders tab, mirroring the "past orders" pattern of DoorDash/Uber Eats.
Favourites & theme	Per-user favourite restaurants; light/dark theme toggle following system by default.
Identity	Anonymous, device-scoped sign-in via Supabase Auth — no login screen required to place an order; an optional name/email profile can be set locally.
Backend	Supabase Postgres. Restaurants, menu items, and promo codes are public read-only reference data; orders, order items, and favourites are scoped to the signed-in (including anonymous) user and enforced by Postgres row-level security, not just client-side filtering.

Honest caveat: payment method selection at checkout is a UI choice only — no money actually moves yet — and order tracking is a client-side timer simulation, not a live feed from a kitchen or a rider's phone. Both are the first things Section 6 addresses.

06

What's expected next

Everything above is the interaction design and data layer. Turning it into a real marketplace means building the operational half: money actually moving, orders actually reaching a kitchen and a rider, and someone on each side able to manage that without touching a SQL console.

Required before launch

CapabilityWhy it's required		
Real payment processing	EcoCash API (or a Zimbabwean aggregator such as Paynow) plus card processing, with server-side payment verification before an order is confirmed.	**Planned**
Real order dispatch	A server-side order state machine, vendor acceptance/rejection, and rider assignment to replace the fixed-length timer simulation.	**Planned**
Live tracking	Real rider location feeding the tracking screen, not scripted stages.	**Planned**
Vendor dashboard	Restaurants and supermarkets need to manage their own menu, hours, and availability, and accept or reject incoming orders, without a developer editing the database.	**Planned**
Rider app	Job queue, navigation, proof of delivery, and payout tracking for couriers.	**Planned**
Push notifications	Order-status updates currently only show as an in-app toast, which is invisible if the app isn't open.	**Planned**
Named accounts	Anonymous per-device identity doesn't carry order history across devices and can't authenticate a vendor or rider.	**Planned**
Cancellations & support	A way to cancel or refund an order and reach a human when something goes wrong.	**Planned**
Admin/ops console	Vendor approval, promo code management, dispute handling — currently all manual, direct-to-database.	**Planned**

Later, once the marketplace is live

• Ratings and reviews for restaurants and individual items.

• Scheduled / pre-orders.

• Multiple saved delivery addresses per customer.

• Loyalty or rewards program.

• Geofenced delivery zones with dynamically calculated fee and ETA, replacing the current fixed per-restaurant fee and time-range strings.

• Expansion beyond Harare to other Zimbabwean cities.

• In-app chat between customer, vendor, and support.

• Multi-currency handling if USD-only pricing stops being sufficient.

07

Information architecture

The primary customer flow, as currently routed:

Home→Restaurant→Cart→Checkout→Tracking→Orders

Login is a secondary, optional flow — an order can be placed without ever visiting it, since identity is anonymous by default.

08

Data model

Six tables today, split cleanly between shared reference data and per-user data:

restaurants

Public, read-only. Name, cuisine, rating, delivery time/fee, tags, brand colour. Currently edited by hand, not by vendors.

menu_items

Public, read-only. Belongs to a restaurant; name, description, price, category, popular flag.

promo_codes

Public, read-only. Percent or flat discount, minimum subtotal.

orders

Owned by the placing user only (row-level security). Customer and delivery details, payment method, full fee breakdown, total.

order_items

Line items for an order, snapshotted at purchase time so later menu changes don't rewrite order history.

favorites

Owned by the user only. A simple user–restaurant join.

09

Tech stack

Client

Flutter / Dart

One codebase targeting iOS, Android, and web.

State

Provider

Cart, app state (auth/theme/favourites), and restaurant data as ChangeNotifiers.

Navigation

go_router

Declarative, URL-based routing — works the same on web and mobile.

Backend

Supabase

Managed Postgres, anonymous auth, and row-level security policies enforced server-side.

Local storage

shared_preferences

Caches cart, theme, and profile so the app works offline and starts fast.

Type

Bricolage Grotesque + Inter

Display and body faces, via Google Fonts.

10

Design principles

• **Zimbabwean-first, not a reskin.** The palette is drawn from the flag and the flame lily — flame red for action, forest green for confirmation, marigold for highlight — not a generic delivery-app template.

• **Familiar interaction, local substance.** The card lists, filters, and checkout flow follow patterns customers already know from global apps, so there's nothing new to learn — but the payment methods, suburbs, and cuisine are Harare's own.

• **Local context from day one.** EcoCash and Harare suburbs are built into the checkout flow now, rather than retrofitted after launch on a payment stack designed for card-first markets.

• **Degrades gracefully.** If Supabase isn't configured, the app falls back to bundled local data — useful for demos, and a deliberate signal that the UI and the backend are cleanly separated.

11

Non-functional expectations

• **Performance:** smooth scrolling and search on mid-range Android hardware, and usable over constrained mobile data — both common conditions in the target market.

• **Reliability:** a placed order must never silently disappear; local and server persistence should agree.

• **Security & privacy:** row-level security enforced in Postgres, not just in the client; only the anon key ships in the app — a service-role key must never be embedded client-side.

• **Accessibility:** legible type scale and adequate contrast in both light and dark themes, tap targets sized for one-handed phone use.

• **Localization:** USD pricing and a Harare suburb list today; the architecture should not assume a single city or currency permanently.

12

Success metrics

To be tracked once the app is live and taking real orders:

• Orders placed per week, and gross merchandise value (GMV).

• Active restaurants and supermarkets on the platform.

• Order completion rate, and actual vs. quoted delivery time.

• Repeat customer rate (7-day and 30-day).

• Rider utilization, once the rider app exists.

13

Roadmap

Phase 0 · Now

Prototype

Full customer UX built against a real database. Static/seeded restaurant data, simulated tracking, no live payments.

Phase 1

MVP launch

Real payment processing, real vendor order acceptance, named accounts, push notifications — the minimum to take a genuine paying order.

Phase 2

Marketplace

Vendor dashboard, rider app, ratings and reviews, admin/ops tooling — the platform runs itself without a developer in the loop.

Phase 3

Scale

Multi-city expansion, loyalty program, scheduled orders, dynamic zone-based fees and ETAs.

14

Open questions & risks

EcoCash integration path

Which aggregator (e.g. Paynow) gives API-level EcoCash access with workable settlement terms for an early-stage business?

Rider supply model

In-house riders, a gig marketplace, or leaning on each vendor's own delivery staff — this decision shapes the entire Phase 2 rider app.

Data protection compliance

What's required for storing customer PII (name, phone, address) for users in Zimbabwe, and does the current Supabase/RLS setup satisfy it.

Anonymous auth ceiling

Whether per-device anonymous identity is sufficient through MVP launch, or whether named accounts need to move up from Phase 1 to a launch blocker.

ZimDash — Product Design DocumentCompanion document: Agreements and arrangements with developers